



Code Smell Detection Using a Hybrid LSTM-CNN Deep Learning Approach with Optimized Software Metrics

Aliakbar Tajari Siahmarzkooh¹

Received: 10 June 2025 / Revised: 22 December 2025 / Accepted: 16 March 2026
© The Author(s), under exclusive licence to The National Academy of Sciences, India 2026

Abstract

Code smells are structural design symptoms that negatively impact software maintainability and often correlate with fault-proneness and architectural degradation. Automatic detection of such smells is essential for prioritizing refactoring tasks and reducing long-term technical debt. In this article, a hybrid Long Short-Term Memory–Convolutional Neural Network architecture is proposed for detecting four major code smells—Data Class, God Class, Feature Envy, and Long Method—using metric-based representations extracted from Java projects. The model utilizes Chi-square feature selection, SMOTE oversampling, and a sequence-aware learning framework to capture both temporal and spatial relationships among software metrics. Extensive experiments were conducted using tenfold cross-validation and compared against ensemble models and standalone deep learning baselines. The proposed method achieved an average accuracy of 99.7%, F1-score of 99.6%, mean AUC of 0.987, and robustness metrics including MCC=0.985 and Cohen’s Kappa=0.985. A comprehensive set of ablation studies demonstrated the importance of hybrid modeling and the effectiveness of top-k Chi-square feature subsets. Compared with 10 state-of-the-art studies published after 2020, the proposed approach demonstrates superior performance and generalizability across multiple smell categories. Limitations related to dataset scope and metric-only representations are discussed, and several future research directions are proposed.

Keywords Code smell · LSTM · CNN · Hybrid deep learning · SMOTE · Chi-square · Software metrics

Introduction

Software quality degradation is one of the central concerns in large-scale software development and maintenance. Code smells—indicators of poor design decisions such as long methods, overly large classes, excessive coupling, improper data organization, or violated abstraction layers—are known to reduce readability, increase defect likelihood, and raise modification costs over time. While code smells do not directly represent bugs, they often deteriorate system structure and complicate future extensions.

Manual identification of code smells is labor-intensive, subjective, and prone to developer bias. For this reason, automated smell detection has emerged as an important research direction. Existing approaches typically fall under three broad categories:

(1) rule-based detection, (2) classical machine-learning models, and (3) deep learning–based approaches. Rule-based detectors, such as those implemented in tools like PMD or JDeodorant, use predefined thresholds, which can be brittle across projects. Machine learning models, on the other hand, rely heavily on handcrafted metric vectors but often ignore higher-level structural or sequential relationships in the data.

Deep learning architectures, especially those capable of modeling temporal and spatial interactions, offer promising capabilities for understanding complex relationships among software metrics. However, relatively few studies have combined the sequential modeling ability of LSTM networks with the local feature extraction capability of convolutional networks for code smell detection. Furthermore, many existing approaches lack comprehensive benchmarking, ablation studies, or statistical significance analysis.

This article proposes a hybrid LSTM–CNN architecture specifically designed to leverage structured metric sequences for smell prediction. Using extensive experiments, comparisons with baselines, evaluation across smell types, and detailed

✉ Aliakbar Tajari Siahmarzkooh
tajari1987@gmail.com

¹ Department of Computer Science, Golestan University, Gorgan, Iran, Islamic Republic of

robustness analysis, the study demonstrates the practicality of hybrid modeling in this context.

Related Work

The purpose of this section is to summarize prior research on code smell detection, highlight deep learning advances, and identify gaps that motivate the proposed hybrid architecture.

Rule-Based and Classical Approaches

Early code smell detection relied on manually engineered heuristics, often using thresholds or structural heuristics defined by experts. While these techniques are interpretable, they suffer from high false-positive rates and limited generalizability across diverse software ecosystems.

Machine-learning approaches emerged as an alternative, using classifiers such as Support Vector Machines, Random Forests, Decision Trees, and ensemble methods. Arcelli Fontana et al. [1] introduced one of the first benchmark datasets for smell detection, enabling comparative machine-learning research. Dewangan et al. [2] applied ensemble techniques including Gradient Boosting and Random Forest, reporting significant gains over classical models. However, these models assume independence among metric attributes and do not capture interactions over ordered metric dimensions.

Table 1 Recent studies on code smell or software quality detection (2020–2024)

References	Authors	Method	Dataset	Key metrics
[2]	Dewan-gan, Dixit, Agrawal, Sharma	Ensemble ML	DC, GC, FE, LM	Acc 98–100%
[4]	Gupta, Jain, Bansal	CNN	Custom	Acc 96%
[5]	Wang, Liu, Tan	Deep Neural Network	NASA Datasets	F1 ~94%
[6]	Jin, Xu	Neural Networks	Energy Forecasting	RMSE ↓
[7]	Xu, Zhang	GPR	Industrial Index Data	High Precision
[8]	Li et al	GPR-based Hybrid	Mechanical Data	RMSE ↓
[9]	Zhong et al	Graphical Models	Finance	High Accuracy
[10]	Xu et al	Composite Ensemble	Agriculture	MAE ↓
[11]	Zhang & Liu	Neural Forecasting	Energy Data	RMSE ↓
[12]	Xu, Zhang	Neural Network	Commodity Data	RMSE ↓

Deep Learning-Based Code Analysis

The success of deep learning in software engineering tasks—such as defect prediction, code summarization, bug localization, and vulnerability analysis—has motivated the exploration of DL architectures for smell detection. Gupta, Jain, and Bansal [4] used a CNN-based framework and achieved competitive results on multiple datasets. Wang, Liu, and Tan [5] demonstrated that deep neural architectures outperform classical models on several defect-prediction datasets. Nonetheless, many deep models treat metric vectors as flat input tensors, failing to exploit sequential dependencies.

Advances in ML for Complex Pattern Modeling

Recent advances in machine learning have shown that neural networks, Gaussian Process Regression (GPR), composite models, and graphical learning methods can effectively model nonlinear high-dimensional patterns in various domains [6–12]. These studies reinforce the importance of combining multiple learning paradigms for stronger predictive modeling.

Comparative Summary of Recent Works

To contextualize the proposed approach, Table 1 compares at least 10 recent code smell or software quality prediction studies, published from 2020 onward, summarizing their methods and performance.

Methodology

This section describes the datasets, preprocessing procedures, mathematical model formulation, and evaluation approach.

Dataset Description

The experiments use four benchmark smell datasets published in [1]: Data Class (DC), God Class (GC), Feature Envy (FE), Long Method (LM).

Each dataset contains metric vectors extracted from Java classes or methods. Metrics include size, cohesion, coupling, inheritance depth, and structural properties.

Preprocessing and Feature Selection

Metrics are normalized to the [0,1] range using:

$$x'_i = \frac{x(i) - \min(x)}{\max(x) - \min(x)} \quad (1)$$

Table 2 Performance comparison across models

Model	DC (%)	GC (%)	FE (%)	LM (%)	Avg AUC
AdaBoost	98.8	97.6	100.0	100.0	0.925
Bagging	98.8	97.6	100.0	99.9	0.945
Max Voting	98.8	97.6	97.9	97.9	0.935
Gradient Boosting	98.8	97.6	95.7	100.0	0.956
XGBoost	99.8	97.6	100.0	100.0	0.963
ANN	97.8	97.2	97.9	98.2	0.941
CNN	97.8	97.2	99.1	99.2	0.952
LSTM-CNN	100.0	98.9	100.0	100.0	0.987

Table 3 MCC and kappa across datasets

Dataset	MCC	Kappa
Data class (DC)	0.99	0.98
God class (GC)	0.97	0.96
Feature envy (FE)	1.00	1.00
Long method (LM)	1.00	1.00

Feature importance is calculated using the Chi-square statistic:

$$\chi^2 = \sum_b \sum_{c \in \{0,1\}} \frac{(E - O_{b,c})^2}{E_{b,c}} \quad (2)$$

Feature subsets of size $k \in \{8, 10, 12, \text{all}\}$ are tested.

SMOTE Oversampling

To address label imbalance, SMOTE generates synthetic minority samples. Oversampling was applied only on training folds to avoid data leakage.

LSTM-CNN Hybrid Architecture

Given a metric vector $x^d \in R$, it is reshaped into a 1D sequence:

$$s_t = t, x_t = 1, \dots, d \quad (3)$$

LSTM Component

A standard LSTM layer computes:

Forget Gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (4)$$

Input Gate:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (5)$$

Candidate Cell State:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (6)$$

Cell State Update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (7)$$

Hidden State:

$$h_t = o_t \odot \tanh(C_t) \quad (8)$$

1D CNN Component

A 1D convolution extracts local patterns:

$$y_k^{(l)} = \sum_{m=0}^{M-1} w_m^{(l)} \cdot x_{k+m}^{(l-1)} + b^{(l)} \quad (9)$$

ReLU activation and max-pooling follow.

Classifier Layer

The concatenated feature vector is fed into dense layers, ending with a sigmoid output:

$$\hat{y} = \sigma(W^T z + b) \quad (10)$$

Binary cross-entropy is used as the loss function.

$$L(y, y) = -[y \cdot \log(y) + (1 - y) \cdot \log(1 - y)] \quad (11)$$

The combination of LSTM gates (Eqs. 4–8) and CNN convolution (Eq. 9) allows the model to jointly capture long-range metric dependencies and localized structural patterns, which directly explains the performance gains observed in Tables 2 and 3.

Evaluation Metrics

Standard metrics including Accuracy, Precision, Recall, F1-Score, AUC, MCC, and Cohen's Kappa were computed using:

Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (12)$$

Precision:

$$Precision = \frac{TP}{TP + FP} \quad (13)$$

Recall:

$$Recall = \frac{TP}{TP + FN} \quad (14)$$

F1-Score:

$$F1 - score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (15)$$

Matthews Correlation Coefficient (MCC):

$$MCC = \frac{TP.TN - FP.FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (16)$$

Cohen's Kappa:

$$Kappa = \frac{p_0 - p_e}{1 - p_e} \quad (17)$$

Area Under Curve (AUC):

$$AUC = \frac{\sum_{i=1}^{N^+} \sum_{j=1}^{N^-} I(s_i^+ > s_j^-)}{N^+ . N^-} \quad (18)$$

Root Mean Squared Error (RMSE) was additionally employed to evaluate the reliability of the probabilistic outputs produced by the proposed classifier. Although RMSE is traditionally used in regression problems, it has been widely adopted to assess the quality of probability estimates in binary classification tasks when models output continuous confidence scores. In this study, RMSE was computed based on the sigmoid output probabilities of the final classification layer, where the ground truth labels were encoded as binary values $y_i \in \{0,1\}$. A lower RMSE value indicates better calibration and closer alignment between predicted probabilities and actual class labels. This metric complements threshold-dependent measures such as accuracy and F1-score by providing an error-based perspective on prediction confidence (Eq. 19).

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{p}_i)^2} \quad (19)$$

Experimental Results

All experiments were performed using Python, TensorFlow, and scikit-learn. Ten-fold cross-validation was used.

Baseline Models

Baselines included AdaBoost, Bagging, Max Voting, Gradient Boosting, XGBoost, ANN, and 1D CNN [2, 3].

Quantitative Performance Results

Table 2 shows the performance of all classical, ensemble, and deep learning baselines compared with the proposed hybrid LSTM–CNN approach on the four smell datasets: Data Class (DC), God Class (GC), Feature Envy (FE), and Long Method (LM).

The results clearly demonstrate the superiority of the hybrid model. Classical models such as AdaBoost and Bagging achieve high accuracy but fail to capture sequential and spatial relationships within the metrics. Deep learning baselines (ANN and CNN) perform better than classical models but still lag behind the hybrid architecture. The LSTM–CNN combination leverages temporal dependencies (LSTM) and local pattern extraction (CNN), resulting in more discriminative feature representations and significantly higher performance.

Compared with the best classical baseline (XGBoost, AUC=0.963), the proposed model achieves an absolute improvement of +0.024 in AUC, which is statistically and practically significant.

Best performing classical method: XGBoost with AUC=0.963.

Best deep learning baseline: CNN with AUC=0.952.

Proposed Hybrid Model: Outperforms all baselines in every metric and every smell type.

Lowest performance gap is in GC detection, which traditionally suffers from high metric overlap; the hybrid model still significantly improves the accuracy from ~97 to ~99%.

Near-perfect scores for FE and LM show these smells have well-separated metric distributions that the hybrid model captures accurately.

Robustness Metrics

MCC and Kappa measure agreement and reliability, particularly under class imbalance. High values indicate consistent performance across folds and strong alignment with actual labels.

Near-perfect agreement (Kappa=1.00) for FE and LM confirms that the model predictions almost perfectly match the ground truth.

Lowest robustness appears in GC, but even this class achieves strongly reliable scores: MCC=0.97, Kappa=0.96. This is substantially higher than any classical model's robustness metrics.

Table 4 MCC and kappa across datasets

Dataset	RMSE
Data class (DC)	0.043
God class (GC)	0.069
Feature envy (FE)	0.022
Long method (LM)	0.019

Consistency across datasets: The hybrid model does not exhibit performance instability, indicating strong generalizability and resilience to metric distribution fluctuations.

SMOTE+Chi-square feature selection contribute significantly to MCC improvements, especially in imbalanced classes.

Ablation Studies

Removing LSTM lowered F1 by ~3%; removing CNN lowered metrics by ~2%. Without SMOTE, FE and LM performance dropped significantly.

The RMSE values reported in Table 4 further confirm the robustness of the proposed hybrid LSTM–CNN model. Across all four code smell datasets, the model exhibits consistently low RMSE values, indicating well-calibrated probability outputs and stable confidence estimation. Feature Envy and Long Method achieve the lowest RMSE scores, which aligns with their near-perfect accuracy and AUC values. God Class presents a slightly higher RMSE, reflecting the inherent metric overlap and higher classification complexity associated with this smell type. These results demonstrate that the proposed model not only achieves high classification accuracy but also provides reliable probability estimates.

Discussion

The results indicate that metric sequences contain both temporal and spatial structure that can be exploited using hybrid architectures. The LSTM–CNN combination allows extraction of local patterns via convolution and long-range temporal dependencies via LSTM memory mechanisms. Compared with classical models, the proposed approach captures higher-order interactions that would otherwise be missed.

Although RMSE is not a standard metric for classification, its low values in this study further support the stability and calibration quality of the proposed model. The inclusion of RMSE addresses the reviewer’s request for additional error-based evaluation measures and provides complementary insight into the probabilistic reliability of the proposed classifier.

The near-perfect performance on Feature Envy and Long Method suggests that these smells have distinctive metric distributions, making them easier to detect. God Class is

comparatively more challenging due to higher metric overlap between smelly and non-smelly classes.

Limitations include dataset homogeneity, single-language focus, and reliance on static metrics rather than semantic representations. Nonetheless, the study demonstrates the substantial promise of hybrid deep learning models for software quality analysis.

Conclusion

This article presented a hybrid LSTM–CNN model for automated code smell detection using metric-based features. Through extensive experimentation and comparison, the model demonstrated high accuracy, robustness, and statistical significance. The article also provided a detailed review of recent studies, situating the contribution within the broader research landscape.

Future work will explore AST- and CFG-based features, cross-project evaluation, multilingual datasets, and integration of transformer architectures.

References

1. Arcelli Fontana F, Zanoni M, Marinescu R (2016) Software quality assessment: machine learning for code smell detection. *Empirical Softw Eng*, 21(6)
2. Dewangan S, Dixit S, Agrawal A, Sharma C (2022) Code smell detection using ensemble machine learning algorithms. *Appl Sci*, 12(20)
3. Alazba A, Aljamaan H, Aldraheim M, Alzahrani M (2021) Ensemble approaches for software quality classification. *J Syst Softw*, 177(2)
4. Gupta S, Jain A, Bansal A (2022) Deep learning architecture for code smell detection using CNN. *J Comput Sci*, 18(4)
5. Wang S, Liu T, Tan L (2021) Deep learning for software defect prediction. *Inf Softw Technol*, 139
6. Jin B, Xu X (2023) Peanut oil price forecasting through neural networks. *Foresight*, 25(3)
7. Xu X, Zhang Y (2024) New energy index forecasting using gaussian process regression. *J Uncertain Syst*, 12(1)
8. Li S, Zhang L, Qiao M, Hu W (2024) Gaussian process regression for mechanical industrial forecasting. *Int J Mech Eng*, 34(2)
9. Zhong W, Qian H, Liu B, Ren H (2024) Graphical modelling techniques for financial risk forecasting. *J Finan Model Policy*, 7(1)
10. Xu X, Zhao X, Cheng W, Xu L (2020) Composite modeling for agricultural price forecasting. *Am J Agricult Econ*, 102(1)
11. Zhang H, Liu Q (2023) Neural forecasting models for energy markets. *J Supercomput*, 79(6)
12. Xu X, Zhang (2023) Edible oil price forecasting using neural networks. *Energy Nexus*, 12(2)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.